

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

AIM: A* Search and Recursive Best-First Search

Implement the Recursive Best-First Search algorithm for the same problem.

INPUT:

```
import heapq
import matplotlib.pyplot as plt
import networkx as nx

graph = {
    'MVLU College': {'Hubtown Solaris': 1.8},
    'Hubtown Solaris': {
        'CD Barfiwala Junction': 1.5,
        'Juhu Lane Junction': 1.6,
        'Azad Nagar Metro': 2.3
    },
    'CD Barfiwala Junction': {'ISKCON Temple': 1.0},
    'Juhu Lane Junction': {'ISKCON Temple': 1.0},
    'Azad Nagar Metro': {'ISKCON Temple': 2.8},
    'ISKCON Temple': {}
}

# (Measure these using Google Maps "Measure Distance")

heuristics = {
    'MVLU College': 2.73,
    'Hubtown Solaris': 2.10,
    'CD Barfiwala Junction': 0.85,
    'Juhu Lane Junction': 0.95,
    'Azad Nagar Metro': 2.55,
    'ISKCON Temple': 0
}

node_positions = {
    'MVLU College': (5,8),
    'Hubtown Solaris': (5,6),
    'CD Barfiwala Junction': (3,4),
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
'Juhu Lane Junction': (5,4),
'Azad Nagar Metro': (7,4),
'ISKCON Temple': (5,1)
}

def a_star_search(graph, heuristics, start, goal):

    priority_queue = [(heuristics[start], start, [start], 0)]

    visited = set()

    while priority_queue:

        f_score, current, path, g_score = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)

        if current == goal:
            return path, g_score

        for neighbor, edge_weight in graph[current].items():

            if neighbor not in visited:

                next_g = g_score + edge_weight
                next_f = next_g + heuristics[neighbor]

                heapq.heappush(
                    priority_queue,
                    (next_f, neighbor, path + [neighbor], next_g)
                )

    return None, float('inf')
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
optimal_path, total_distance = a_star_search(
    graph,
    heuristics,
    'MVLU College',
    'ISKCON Temple'
)

print("Optimal Path Discovered:")
print(" -> ".join(optimal_path))

print(f"\nTotal Road Distance : {total_distance} km")

G = nx.DiGraph()

for node, neighbors in graph.items():
    for neighbor, weight in neighbors.items():
        G.add_edge(node, neighbor, weight=weight)

plt.figure(figsize=(11,8))

path_edges = list(zip(optimal_path, optimal_path[1:]))

normal_edges = [
    edge for edge in G.edges()
    if edge not in path_edges
]

nx.draw_networkx_nodes(
    G,
    node_positions,
    node_color='lightblue',
    node_size=2500
)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

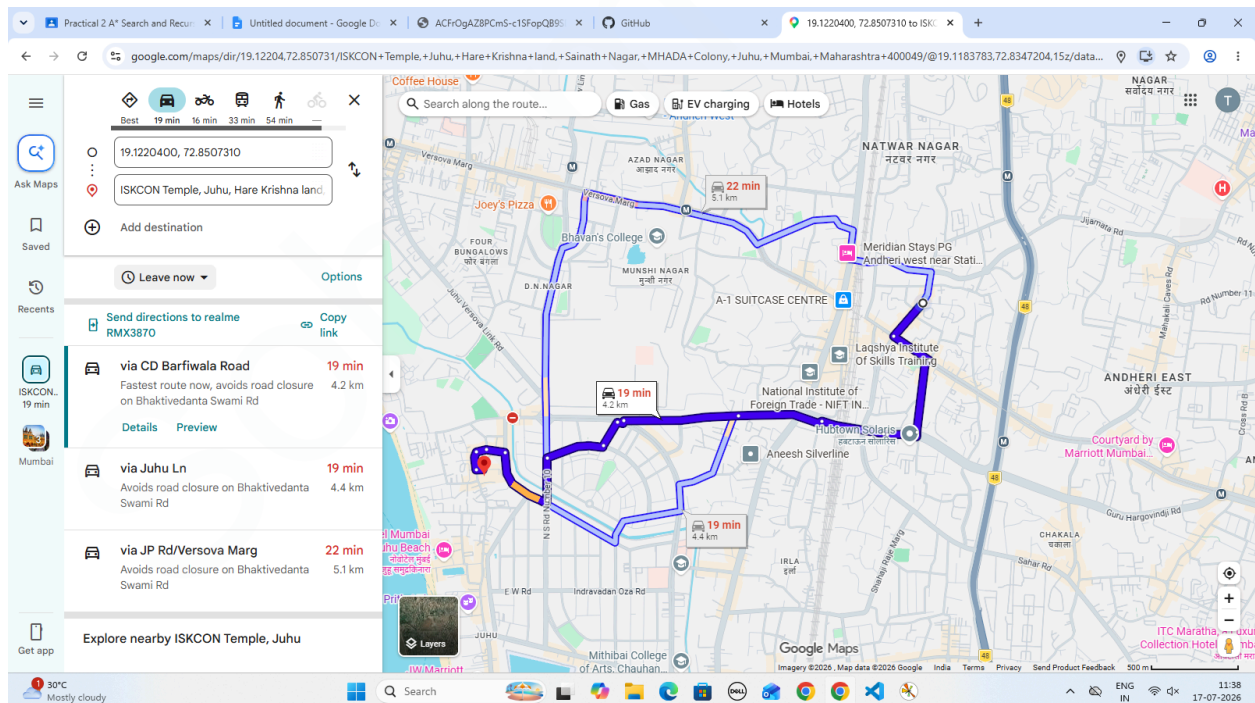
```
nx.draw_networkx_edges(  
    G,  
    node_positions,  
    edgelist=normal_edges,  
    edge_color='gray',  
    arrows=True,  
    width=2  
)  
  
nx.draw_networkx_edges(  
    G,  
    node_positions,  
    edgelist=path_edges,  
    edge_color='darkorange',  
    arrows=True,  
    width=4  
)  
  
node_labels = {  
    node: f"{node}\nh(n)={heuristics[node]}"  
    for node in G.nodes()  
}  
  
nx.draw_networkx_labels(  
    G,  
    node_positions,  
    labels=node_labels,  
    font_size=9,  
    font_weight='bold'  
)  
  
edge_labels = nx.get_edge_attributes(G, 'weight')  
  
formatted_edge_labels = {  
    edge: f"{weight} km"  
    for edge, weight in edge_labels.items()  
}
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
nx.draw_networkx_edge_labels(  
    G,  
    node_positions,  
    edge_labels=formatted_edge_labels,  
    font_color='red'  
)  
  
plt.title(  
    "A* Search\nMVLU College to ISKCON Temple, Juhu",  
    fontsize=14  
)  
  
plt.axis("off")  
plt.tight_layout()  
plt.show()
```

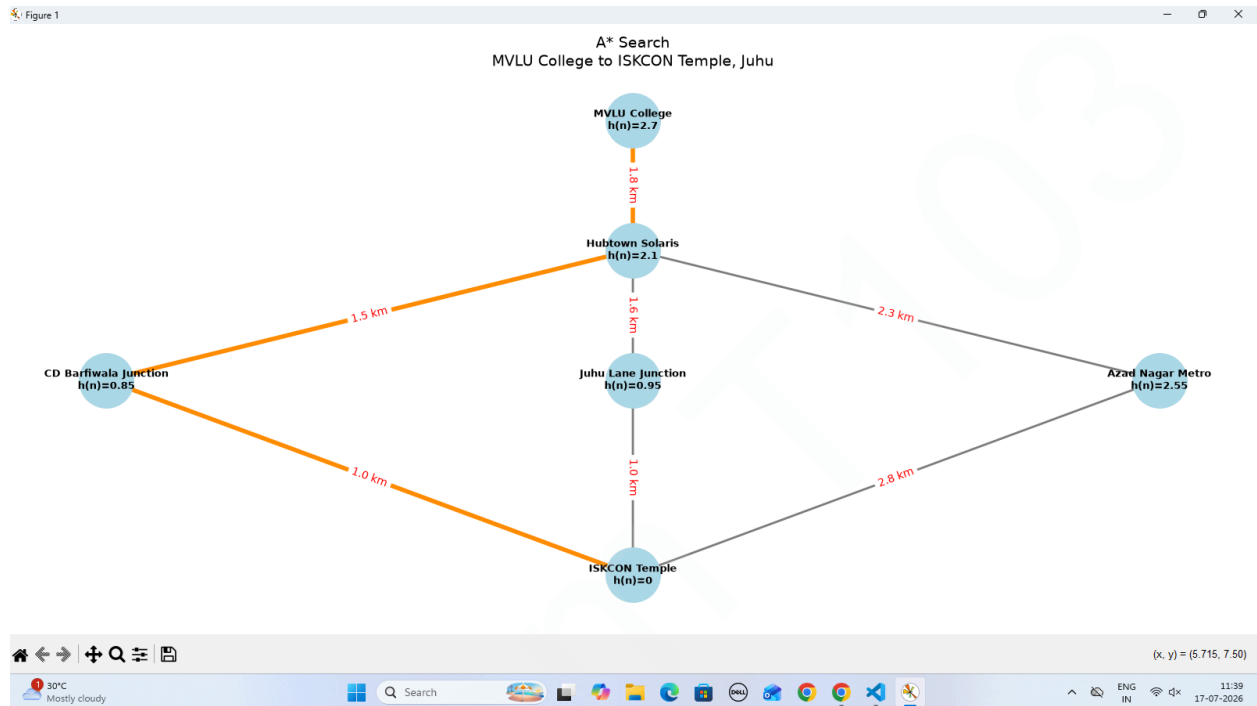
MAP



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

Output



Input for recursive

```
import matplotlib.pyplot as plt

#
# 1. ROAD NETWORK (Marol Naka -> Karjat)
#

graph = {
    'Marol Naka': {
        'Vikhroli': 5.9,
        'Airport': 4.0
    },
    'Vikhroli': {
        'Marol Naka': 5.9,
        'Airport': 1.0
    },
    'Airport': {
        'Marol Naka': 4.0,
        'Vikhroli': 1.0
    }
}
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
# Route 1 : Via Vikhroli -> Sion Panvel Highway
'Vikhroli': {
    'Sion Panvel Hwy': 6.1
},

'Sion Panvel Hwy': {
    'Panvel': 44.6
},

# Route 2 : Via Airport -> Chembur -> Vashi
'Airport': {
    'Chembur': 13.8
},

'Chembur': {
    'Vashi': 30.9,
    'Atal Setu': 35.7
},

'Vashi': {
    'Panvel': 21.6
},

# Route 3 : Via Atal Setu
'Atal Setu': {
    'Panvel': 15.8
},

'Panvel': {
    'Karjat': 21.6
},

'Karjat': {}
}

#
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
# Straight-line heuristic h(n) to Karjat
#
-----

heuristics = {
    'Marol Naka': 54,
    'Vikhroli': 48,
    'Airport': 55,
    'Airport MTHL': 54,
    'Sion Panvel Hwy': 42,
    'Chembur': 43,
    'Vashi': 35,           # Added missing node to prevent KeyError
    'Atal Setu': 27,
    'Panvel': 18,
    'Karjat': 0
}

#
-----

# Coordinates for plotting
#
-----

coords = {
    'Marol Naka': (2, 8),
    'Kurla': (1, 6),
    'Airport': (1, 6),           # Added missing node coordinate mapping
    'Powai': (4, 7),
    'Vikhroli': (4, 7),         # Added missing node coordinate mapping
    'Sion': (6, 6),
    'Sion Panvel Hwy': (6, 6),  # Added missing node coordinate mapping
    'Chembur': (1, 4),
    'Airoli': (4, 5),
    'Vashi': (2, 2),
    'MTHL': (6, 3),
    'Atal Setu': (6, 3),        # Added missing node coordinate mapping
    'Panvel': (5, 1),
    'Chowk': (8, 0),
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
'Karjat': (11, -1)
}

#
-----
# RBFS ALGORITHM
#
-----

def rbfs_search(start, goal):
    success, path, cost, _ = rbfs(
        start,
        goal,
        g=0,
        f_limit=float('inf'),
        path=[start]
    )
    return path, cost

def rbfs(node, goal, g, f_limit, path):
    if node == goal:
        return True, path, g, g

    neighbors = graph[node]

    if not neighbors:
        return False, [], 0, float('inf')

    successors = []

    for neighbor, distance in neighbors.items():
        if neighbor not in path:
            next_g = g + distance
            next_f = max(
                next_g + heuristics[neighbor],
                g + heuristics[node]
            )
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
        successors.append(
            [next_f, neighbor, next_g]
        )

    if not successors:
        return False, [], 0, float('inf')

    while True:
        successors.sort(key=lambda x: x[0])
        best = successors[0]

        if best[0] > f_limit:
            return False, [], 0, best[0]

        alternative = successors[1][0] if len(successors) > 1 else
float('inf')

        success, result_path, total_g, returned_f = rbfs(
            best[1],
            goal,
            best[2],
            min(f_limit, alternative),
            path + [best[1]]
        )

        best[0] = returned_f

        if success:
            return True, result_path, total_g, returned_f

#
-----
# RUN
#
-----

path, total_distance = rbfs_search("Marol Naka", "Karjat")
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
print("\nOptimal RBFS Path:\n")
print(" -> ".join(path))
print(f"\nTotal Distance = {total_distance} km")

#
-----

# GRAPH VISUALIZATION
#
-----

plt.figure(figsize=(12, 7))

# Draw Edges
for node, neighbors in graph.items():
    x1, y1 = coords[node]
    for neighbor, dist in neighbors.items():
        x2, y2 = coords[neighbor]

        is_path = (
            node in path and
            neighbor in path and
            path.index(neighbor) == path.index(node) + 1
        )

        color = '#2ecc71' if is_path else '#b0b0b0'
        width = 3 if is_path else 1.5

        plt.plot(
            [x1, x2],
            [y1, y2],
            color=color,
            linewidth=width
        )

        plt.text(
            (x1 + x2) / 2,
            (y1 + y2) / 2,
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
        f"{dist} km",
        color='red',
        fontsize=9
    )

# Draw Nodes
for node, (x, y) in coords.items():
    # Only render nodes that are actively part of the defined route
    network_graph
    if node in graph or any(node in nbrs for nbrs in graph.values()):
        node_color = '#f39c12' if node in path else '#3498db'

    plt.scatter(
        x,
        y,
        s=1200,
        color=node_color,
        edgecolors='black',
        zorder=3
    )

    plt.text(
        x,
        y,
        f"{node}\nh={heuristics[node]}",
        ha='center',
        va='center',
        fontsize=8,
        color='white',
        fontweight='bold'
    )

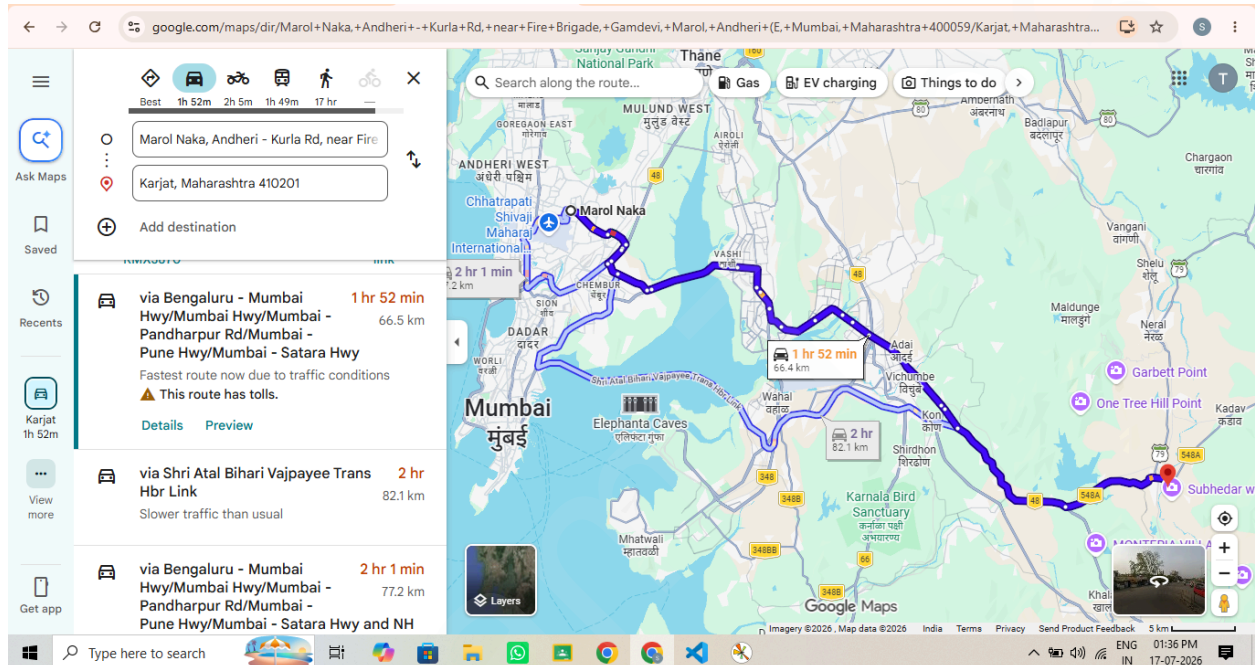
plt.title(
    f"Recursive Best-First Search\nMarol Naka → Karjat\nTotal Distance =
{round(total_distance, 2)} km",
    fontsize=14,
    fontweight='bold'
)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
plt.axis('off')  
plt.show()
```

Map for recursive



Output

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

Figure 1

